

# Le Mot Juste

Jeu *Motus* en architecture microservices

Master 2 MIAGE SITN — Applications Web orientées Services

Enseignant : M. Mouloud Menceur

**Binôme : Omar Soliman • Abderrahmane Tsouli**

Dépôt GitHub : <https://github.com/omarsoliman02/LeMotJuste>

Démo en ligne : <https://lemotjuste.duckdns.org>

Juillet 2026

## 1 Présentation

"Le Mot Juste" est notre implémentation du jeu **Motus** sous forme de microservices Spring Boot. Le joueur démarre une partie, reçoit un mot mystère dont il connaît la longueur et la première lettre, puis dispose de **6 essais** pour le deviner. À chaque proposition, le jeu indique lettre par lettre ce qui est bien placé, mal placé ou absent, un peu comme dans Wordle. Les résultats sont historisés et alimentent des statistiques ainsi qu'un classement des joueurs.

Nous avons découpé l'application par domaine métier : un service pour les joueurs, un pour la logique du jeu, un pour les scores. Un point d'entrée unique, la *gateway*, expose l'ensemble au client, qui ne parle jamais directement à un service. Une page web permet de jouer entièrement depuis le navigateur, avec une interface que nous avons voulue proche de Wordle dans l'esprit.

## 2 Architecture

Le système repose sur quatre services Spring Boot et une instance PostgreSQL partagée. La communication entre services se fait en OpenFeign, de façon synchrone, et chaque service reste propriétaire de sa propre base : aucun ne va lire ou écrire directement dans les tables d'un autre, tout passe par une API REST ou un appel Feign.

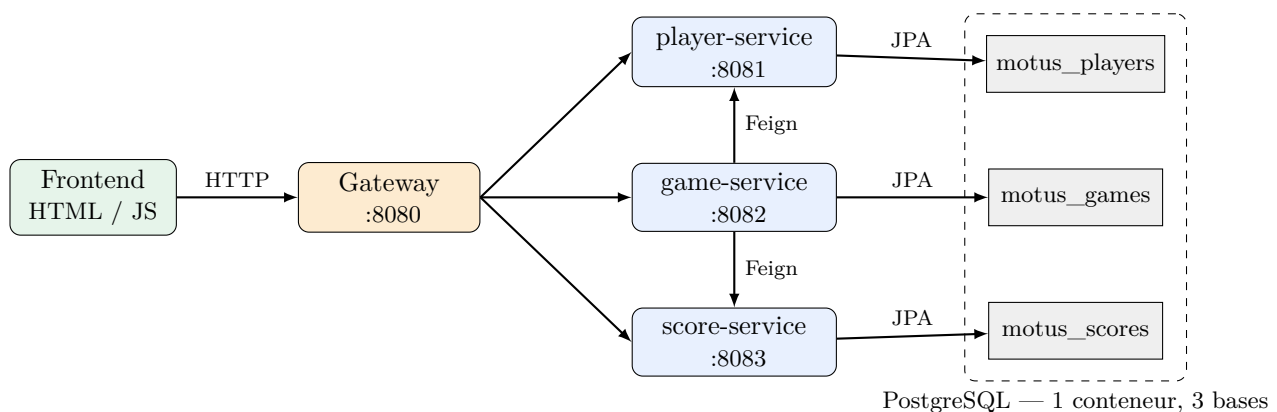


FIGURE 1 — Schéma d'architecture. La gateway route `/api/players`, `/api/games` et `/api/scores`. `game-service` appelle `player-service` pour vérifier que le joueur existe et `score-service` pour enregistrer le résultat.

Le **gateway** (port 8080) est le seul point d'entrée de l'application : c'est lui qui route les requêtes vers le bon service et qui gère les autorisations CORS pour le frontend. Le **player-service** (8081)

s'occupe de l'enregistrement et de la gestion des joueurs, avec sa base `motus_players`. Le **game-service** (8082) porte toute la logique du jeu : tirage du mot mystère, validation d'une proposition, calcul lettre par lettre, gestion du dictionnaire ; il stocke ses parties dans `motus_games`. Enfin le **score-service** (8083) conserve l'historique des parties terminées, calcule les statistiques et le classement, dans `motus_scores`.

### 3 Diagramme de classes "métier"

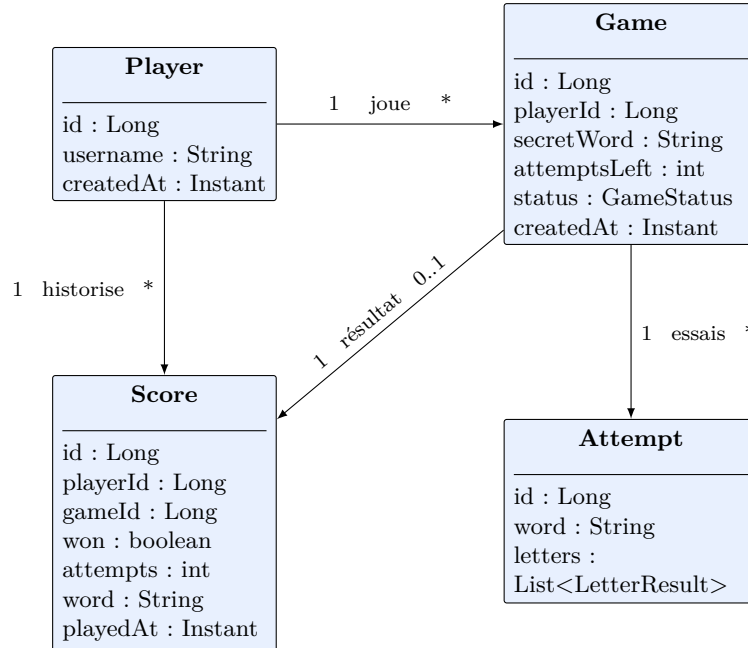


FIGURE 2 – Entités métier et relations. `Game.secretWord` n'est jamais renvoyé au client tant que la partie est `IN_PROGRESS`. `GameStatus` vaut `IN_PROGRESS`, `WON`, `LOST` ou `ABANDONED` ; seules les parties `WON/LOST` produisent un `Score`, une partie abandonnée n'en a aucun (relation 0..1). L'historique des essais (`Attempt`) est calculé à la volée et conservé côté client, jamais persisté.

### 4 Choix techniques

| Élément            | Choix                                       |
|--------------------|---------------------------------------------|
| Langage            | Java 21 (LTS)                               |
| Framework          | Spring Boot 4.0.7                           |
| Spring Cloud (BOM) | 2025.1.2 (Gateway + OpenFeign)              |
| Build              | Maven, un projet par service                |
| Persistence        | Spring Data JPA + PostgreSQL                |
| Conteneurisation   | Dockerfile multi-stage + docker-compose     |
| Orchestration      | Kubernetes / MiniKube                       |
| Frontend           | HTML + JS vanilla, style Wordle, responsive |
| Production         | Caddy (reverse proxy + TLS) sur une VM      |

TABLE 1 – Stack technique.

Nous avons découpé l'application par domaine métier (joueurs, jeu, scores) plutôt que par couche technique, ce qui limite le couplage entre services et permet à chacun de nous de travailler sur un service sans attendre l'autre. Une seule instance PostgreSQL héberge les trois bases logiques :

trois conteneurs séparés n'auraient rien apporté vu la taille du projet. Pour la communication inter-services, nous avons préféré OpenFeign en synchrone à une solution par messages, car les deux flux qui en ont besoin (démarrer une partie, enregistrer un score) sont simples et n'ont pas besoin d'être découplés dans le temps.

Le frontend est volontairement une page statique en HTML et JavaScript pur, sans framework ni étape de build, qui n'appelle que la gateway. Ce choix nous a permis d'itérer très vite dessus pendant tout le projet. Enfin, nous sommes restés sur un périmètre réduit assumé : pas d'Eureka, pas de Config Server, pas de broker de messages, parce que le projet ne justifiait pas cette complexité et que nous avons préféré du code lisible à une architecture qui aurait fait plus "microservices sur le papier" que réellement utile ici.

## 5 API REST

Toutes les routes passent par la gateway (port 8080). Les erreurs répondent dans un format JSON homogène (`timestamp`, `status`, `error`, `message`).

| Méthode | Chemin                               | Rôle                                            |
|---------|--------------------------------------|-------------------------------------------------|
| POST    | <code>/api/players</code>            | Créer un joueur (201 ; 409 si pris)             |
| GET     | <code>/api/players/{id}</code>       | Lire un joueur                                  |
| POST    | <code>/api/games</code>              | Démarrer une partie (taille de grille au choix) |
| POST    | <code>/api/games/{id}/guess</code>   | Proposer un mot (calcul des lettres)            |
| POST    | <code>/api/games/{id}/abandon</code> | Abandonner une partie en cours                  |
| GET     | <code>/api/games/{id}</code>         | État d'une partie (sans le mot mystère)         |
| GET     | <code>/api/games</code>              | Lister toutes les parties (vue admin)           |
| POST    | <code>/api/scores</code>             | Historiser un résultat (appelé en Feign)        |
| GET     | <code>/api/scores?playerId=</code>   | Historique d'un joueur                          |
| GET     | <code>/api/scores/leaderboard</code> | Classement                                      |

TABLE 2 – Principaux points d'entrée de l'API.

## 6 Algorithme Motus

Le cœur du jeu est le calcul, lettre par lettre, d'une proposition par rapport au mot secret. La difficulté vient des lettres en double : il faut éviter qu'un mot avec deux fois la même lettre ne fasse apparaître trop de lettres "présentes". Nous avons résolu ça en deux passes. La première marque **CORRECT** chaque lettre bien placée et compte, dans une table de fréquences, les lettres du mot secret qui restent disponibles. La seconde passe reprend les positions non encore résolues : si la lettre proposée est encore disponible dans la table, elle devient **PRESENT** et on décrémente le compteur, sinon elle est **ABSENT**.

Un exemple aide à comprendre pourquoi les deux passes sont nécessaires. Avec le mot secret **ALLER** et la proposition **LELLE**, les deux L de la proposition ne peuvent pas être tous les deux signalés "présents" puisque **ALLER** n'en contient que deux au total, et un seul n'est pas déjà à sa place. La première passe consomme d'abord ce qui est réellement à sa place, la seconde ne distribue que ce qu'il reste. Toutes les chaînes sont normalisées en majuscules et sans accents avant comparaison, ce qui rend le jeu insensible à la casse.

Côté validation, une proposition est rejetée avant tout calcul si sa longueur ne correspond pas au mot mystère ou si le mot n'existe pas dans le dictionnaire ; dans ce cas l'essai n'est pas décompté, avec un message clair pour le joueur. Comme dans Wordle, nous utilisons deux listes de mots : une liste restreinte de mots courants dans laquelle le mot mystère est tiré au sort, et une liste beaucoup plus large qui sert uniquement à valider si une proposition est un mot du dictionnaire. Cela permet

d'accepter des mots plus rares ou familiers en proposition, tout en gardant des solutions toujours reconnaissables.

## 7 Frontend

Le frontend est une page statique (HTML, CSS, JavaScript, sans framework) qui ne parle qu'à la gateway et dont l'interface reprend les codes de Wordle : grille de tuiles colorées, clavier virtuel, police épaisse. Nous l'avons aussi rendue responsive, en gardant la même largeur de colonne centrée sur téléphone comme sur ordinateur, avec un en-tête qui passe sur deux lignes et des tuiles dont la taille s'ajuste au nombre de lettres pour ne jamais déborder de l'écran.

Sur l'écran d'accueil, avant de choisir son pseudo, une rangée de tuiles animées et trois pictogrammes résument en un coup d'œil comment jouer. Un bouton "Règles" dans l'en-tête ouvre à tout moment une fenêtre récapitulant les règles, sans quitter la partie en cours. Avant de lancer une partie, le joueur choisit la taille de la grille (de 4 à 10 lettres, ou "Auto" pour un tirage aléatoire comme au départ) : ce choix est transmis à `POST /api/games` et validé côté serveur.

Une question s'est posée en cours de route : que devient une partie que le joueur n'a jamais terminée, par exemple parce qu'il change d'avis sur la taille de grille ou qu'il ferme l'onglet ? Au départ, elle restait simplement "en cours" indéfiniment, ce qui polluait les statistiques. Nous avons ajouté un vrai statut `ABANDONED` et un endpoint dédié (`POST /api/games/{id}/abandon`) qui marque la partie comme abandonnée sans lui attribuer de score. Le frontend l'appelle automatiquement quand le joueur clique sur "Annuler", quand il change de compte, et même à la fermeture de l'onglet grâce à `navigator.sendBeacon`, qui reste fiable à ce moment précis contrairement à un appel réseau classique.

La vue **admin**, accessible depuis l'en-tête, est protégée par un mot de passe saisi sur un écran de connexion dédié. C'est une protection côté client, pas une vraie authentification serveur : nous n'avons volontairement pas ajouté Spring Security, qui restait hors périmètre pour ce projet. Une fois connecté, l'admin retrouve six indicateurs synthétiques (nombre de joueurs, de parties, de parties en cours, taux de victoire global, parties perdues, parties abandonnées), le classement des joueurs, et deux tableaux détaillés (parties et scores) filtrables par nom de joueur et par période. Ce filtrage se fait entièrement côté client puisque les données sont déjà chargées, sans appel réseau supplémentaire.

Quelques finitions viennent compléter l'ensemble : une favicon reprenant une tuile verte du jeu plutôt qu'un logo générique, et un titre cliquable dans l'en-tête qui ramène à l'écran principal, comme le ferait le logo d'une application, sans pour autant déconnecter le joueur ni abandonner une partie en cours.

## 8 Déploiement et test de l'application

Le projet peut être testé de trois façons, de la plus rapide à la plus complète.

**1. Depuis un navigateur, sans rien installer.** L'application tourne en continu sur une VM que nous administrons, accessible à cette adresse :

<https://lemotjuste.duckdns.org>

C'est le moyen le plus simple de tester le projet : il suffit d'ouvrir le lien pour créer un joueur et jouer une partie. En coulisses, Caddy sert le frontend en statique et fait suivre les appels `/api/**` vers la gateway, qui elle-même route vers les trois services ; le tout tourne en Docker Compose sur la VM avec redémarrage automatique en cas de reboot. Le détail de cette configuration est dans `deploy/README.md` du dépôt.

**2. En local, avec Docker Compose.** C'est la façon la plus pratique de tester le code source directement, par exemple après un `git clone` :

```
docker compose up --build      # démarre Postgres, les 3 services et la gateway
./serve.sh                    # sert le frontend sur http://localhost:5500
```

On peut aussi vérifier l'API directement en ligne de commande :

```
curl -X POST localhost:8080/api/players -H 'Content-Type: application/json' \
  -d '{"username":"alice"}'
curl -X POST localhost:8080/api/games -H 'Content-Type: application/json' \
  -d '{"playerId":1,"wordLength":6}'
curl -X POST localhost:8080/api/games/1/guess -H 'Content-Type: application/json' \
  -d '{"word":"cheval"}'
```

La procédure complète, avec les commandes pour lancer un service seul ou faire tourner les tests, figure dans le `README.md` du dépôt.

**3. Sur un cluster Kubernetes local (MiniKube).** Pour observer l'aspect orchestration du projet plutôt que le simple assemblage de conteneurs, les mêmes images tournent en **Deployment** et **Service** Kubernetes, avec **Secret** pour les identifiants PostgreSQL et **ConfigMap** pour les URLs inter-services :

```
kubectl apply -k k8s/
kubectl get pods -n lemotjuste -w
```

La gateway est exposée en **NodePort**, et un guide pas à pas (schémas, script de démonstration incluant l'arrêt volontaire d'un pod pour observer le redémarrage automatique) se trouve dans `docs/kubernetes-guide.md`.

## 9 Bilan

**Réussites.** Nous sommes contents du résultat : un découpage microservices cohérent, avec le même patron repris pour chaque service, ce qui rend le code prévisible d'un service à l'autre. Fixer le contrat d'API tôt nous a permis de travailler chacun de notre côté sans se marcher dessus. L'algorithme de calcul des lettres tient la route face aux doublons, ce qu'on a pu vérifier avec des tests unitaires plutôt qu'à l'œil. Et l'interface, qu'on a pas mal retravaillée au fil des retours, ressemble maintenant à une vraie petite application plutôt qu'à une simple démo technique.

**Difficultés.** La gestion des lettres en double a été le point le plus subtil du projet : notre première version signalait parfois trop de lettres "présentes", d'où le passage à l'approche en deux passes décrite plus haut. Spring Boot 4 et Spring Cloud Gateway en version réactive nous ont aussi pris du temps à configurer correctement, tout comme la gestion des pannes entre services (on enregistre le score en "best-effort" : si score-service est indisponible, la partie se termine quand même). Une autre difficulté, plus tardive, a été de réaliser que les parties jamais terminées s'accumulaient silencieusement en base sans qu'on l'ait prévu au départ, ce qui nous a poussés à revoir le cycle de vie d'une partie avec le statut **ABANDONED**. Enfin, le dictionnaire initial était trop restreint et refusait des mots français pourtant corrects, ce que nous avons corrigé en le remplaçant par deux listes distinctes, l'une pour les solutions, l'autre plus large pour la validation.

**Ce que nous avons appris.** Au-delà de Spring Cloud Gateway et OpenFeign, ce projet nous a surtout appris à penser une application en plusieurs services qui doivent rester cohérents entre eux sans jamais partager leur base de données. On retient aussi la conteneurisation multi-stage avec Docker, le déploiement sur MiniKube, et la mise en production réelle derrière un reverse proxy, qui n'est pas tout à fait la même chose que de faire marcher un projet en local.

**Ce que nous avons le plus et le moins aimé.** On a aimé pouvoir faire évoluer chaque service indépendamment sans craindre de casser les autres, et voir l'application progressivement prendre forme jusqu'à devenir quelque chose qu'on a plaisir à montrer. En contrepartie, déboguer un échange entre deux services demande souvent d'aller regarder les logs des deux côtés à la fois, ce qui est plus long qu'une simple pile d'appels dans une application classique.